Project 2: Explainability

Human-Centric Artificial Intelligence

For this project, we work with the Palmer Penguins dataset. It contains eight features:

- `species`
- `sex`
- `bill_length_mm`
- `flipper_length_mm`
- `island`
- `year`
- `bill_depth_mm`
- `body_mass_g`

The target feature is `species`, which is a categorical variable with three values: *Adelie*, *Gentoo* and *Chinstrap*. The easiest way to obtain the dataset in Python is using the `palmerpenguins` package.

1 Interpretability and model complexity

In the lecture, we saw that to promote interpretability, we optimize

$$\arg\min_f \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i) + \lambda\Omega(f), \tag{1}$$

where ℓ is some loss function and Ω is a complexity measure.

Task 1

Fit a decision tree on the dataset. Implement a user interface that shows the resulting tree and displays its test accuracy as well as the number of leaves.

Task 2

Train several models with varying degrees of regularization. Then, extend the interface by a slider (λ) that corresponds to regularization parameter in equation 1. Note that λ is different from the parameter used to control the regularization during model fitting. The interface should show the model corresponding to the maximizer of

$$\text{acc}_{\text{test}} - \lambda\Omega(f),$$

where acc_{test} is the accuracy of the model when evaluated on the test dataset. When f is a decision tree, $\Omega(f)$ refers to the number of leaves.

Note: If you are using the `DecisionTreeClassifier` from `scikit-learn`, you can use `max_leaf_node` for regularization.

Task 3

Repeat the same with logistic regression. Choose a suitable complexity measure Ω.

2 Counterfactual explanations

To generate counterfactual explanations, we can implement the following method:

- Randomly sample N points locally around $\mathbf{x}$ and check whether the prediction has the desired class.
- Rank those that have the desired class by their MAD-weighted L^1-distance (see lecture 5) to $\mathbf{x}$
- Display the best k counterfactuals to the user.

Pay attention to the type of data. If it is decimal, you can proceed as above, but what about binary or categorical data? Think of an appropriate way to noise these kinds of features as well.

If no counterfactuals are found, try increasing N and/or altering the variance of the random sampling procedure for some of the features, possibly in an iterative fashion.

Task 4

Extend the user interface by a "Counterfactual" region. Here, the user can select an example $\mathbf{x}$ from the dataset, as well as a target label, and be shown counterfactual examples generated in the above fashion.

Ensure that the interface is linked to the ones in tasks 1-3. In other words: The user should be able to freely select the model class (tree or logistic regression) and sparsity (λ) to determine from which model the counterfactuals are generated.

3 Global Model-Agnostic Methods

Task 5

Finally, extend the interface by a "Feature Effect Plots" region. As for the counterfactuals, it should be linked to the selected model type and selected value of λ.
In this region, the user should be able to select one of the four numerical features and view both a PDP and ALE plot for this feature's effect on the classification probability of each species. Therefore, each plot should contain three curves.

The code for the computation of the PDP and ALE values should be written by you, i.e., do not use a library for them. For ALE, you will need partial derivatives. For which model can you compute them exactly? For which do you have to use a discretization?